

End-to-End ML/DL Project: Salient Object Detection

Project #3: End-to-End ML/DL Project

Student Name: Ardonesa Kaliqanaj

Dataset: ECSSD

Framework: PyTorch

Environment: Google Colab with T4 GPU

1. Introduction / Background

Salient Object Detection (SOD) is a computer vision task that focuses on finding the most visually important object or region in an image. Instead of detecting and classifying every object in a scene, SOD predicts a pixel-level mask for the dominant region that naturally attracts human attention.

The goal of this project was to build an end-to-end Salient Object Detection system from scratch using machine learning and deep learning concepts. The full system includes dataset preparation, preprocessing, augmentation, CNN model design, training, evaluation, visualization, and demo inference.

In this project, I used a CNN encoder-decoder model built from scratch in PyTorch. No pretrained models, pretrained weights, or pretrained feature extractors were used. The model takes an RGB image as input and predicts a one-channel saliency mask. The predicted mask can also be overlayed on the original image to make the result easier to interpret.

This project helped me understand the full deep learning workflow, not only the model itself. A large part of the work was making sure that the dataset was organized correctly, image-mask pairs matched, the DataLoader worked, the model output had the correct shape, and the results could be evaluated both visually and numerically.

2. Dataset

For this project, I used the ECSSD dataset. ECSSD is a public Salient Object Detection dataset that contains semantically meaningful images and pixel-level saliency masks. I chose ECSSD because it is a good beginner-friendly dataset: it is not too large, but it is still realistic enough for training and evaluating a saliency detection model.

The dataset contains 1000 image-mask pairs. Each image has a corresponding ground-truth saliency mask. The mask shows which region of the image is considered salient. This made ECSSD suitable for supervised learning because the model can learn by comparing its predicted mask with the ground-truth mask.

Before training, I inspected the dataset manually in Google Colab. I checked that the image files and mask files matched correctly by filename. I also visualized several image-mask pairs to confirm that the masks were black and white saliency masks and that they corresponded to the correct images.

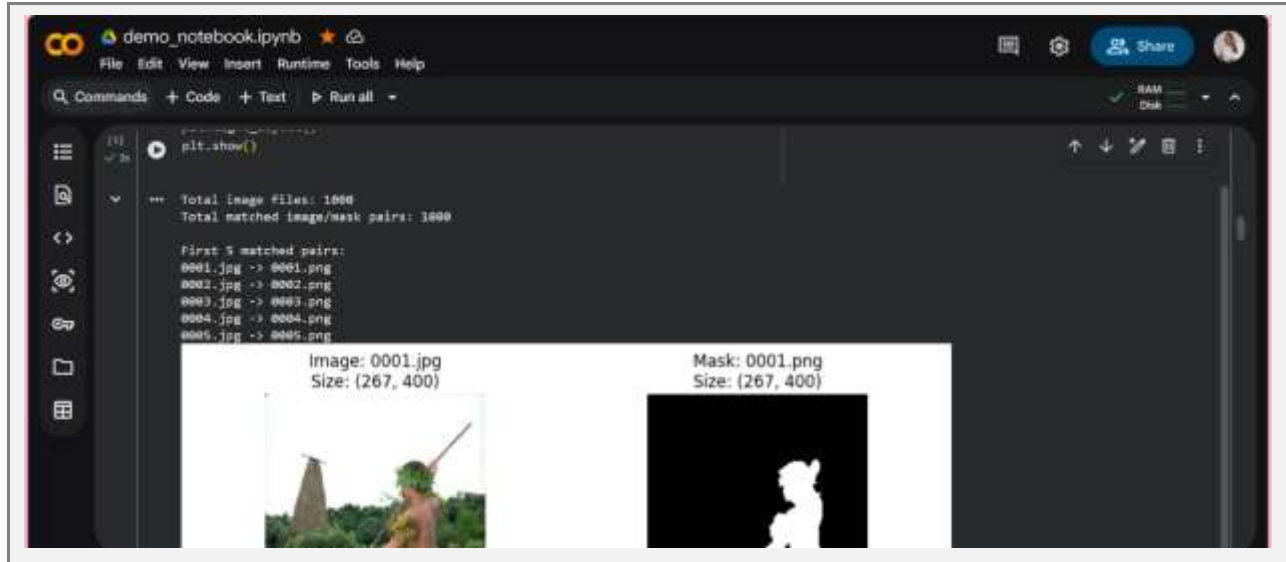


Figure 1. Example ECSSD images and their corresponding ground-truth masks.

The dataset inspection step was important because segmentation projects depend heavily on correct image-mask alignment. If an image is paired with the wrong mask, the model will learn incorrect relationships. In this project, all 1000 image-mask pairs were matched successfully.

3. Preprocessing and Data Augmentation

All images and masks were resized to 128x128 pixels. I used this image size because it keeps the project manageable while still preserving enough visual information for the model to learn the general salient regions. Larger image sizes such as 224x224 could also be used, but they require more memory and training time.

The RGB image pixel values were normalized to the 0-1 range. This means that instead of using raw pixel values from 0 to 255, each pixel value was divided by 255. Normalization helps the model train more smoothly because the input values are in a smaller and consistent range.

The ground-truth masks were converted to single-channel masks. The mask values were also normalized to 0-1 and then thresholded into binary mask values. This matches the project goal because the output is a one-channel saliency mask.

The dataset was split into training, validation, and testing sets using the required 70%, 15%, and 15% split. This produced 700 training image-mask pairs, 150 validation pairs, and 150 test pairs.

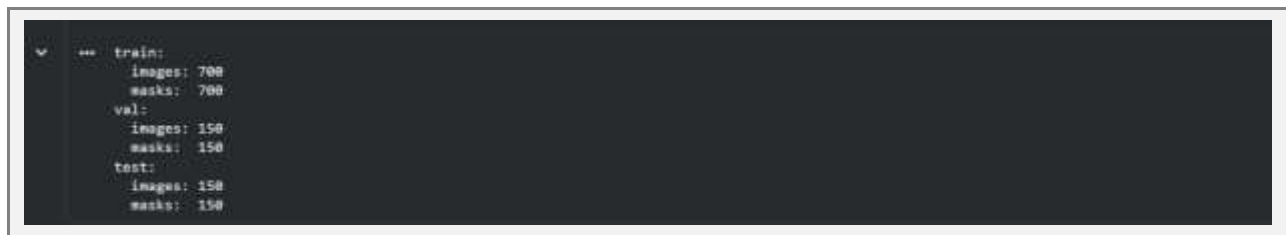
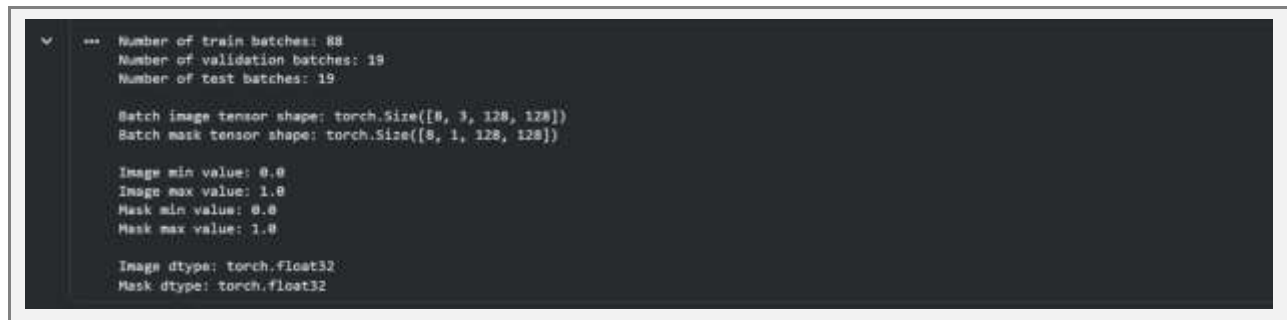


Figure 2. Dataset split showing 700 training pairs, 150 validation pairs, and 150 test pairs.

During training, I applied basic data augmentation to make the model more robust. The augmentations included horizontal flipping, random cropping, and brightness variation. These augmentations were applied only to the training data. The validation and test sets were not augmented because they should represent stable data for measuring performance.

I also tested the PyTorch DataLoader before training. One batch had the expected image tensor shape of [8, 3, 128, 128] and mask tensor shape of [8, 1, 128, 128]. The image and mask values were both in the 0-1 range.

A screenshot of a Jupyter Notebook cell with a dark background and light-colored text. The output shows the configuration of a PyTorch DataLoader, including the number of batches for training, validation, and testing, as well as the shapes and normalized values of the image and mask tensors in a batch. The text is as follows:

```
Number of train batches: 88
Number of validation batches: 19
Number of test batches: 19

Batch image tensor shape: torch.Size([8, 3, 128, 128])
Batch mask tensor shape: torch.Size([8, 1, 128, 128])

Image min value: 0.0
Image max value: 1.0
Mask min value: 0.0
Mask max value: 1.0

Image dtype: torch.float32
Mask dtype: torch.float32
```

Figure 3. DataLoader output showing correct image and mask tensor shapes and normalized values.

4. Model Architecture

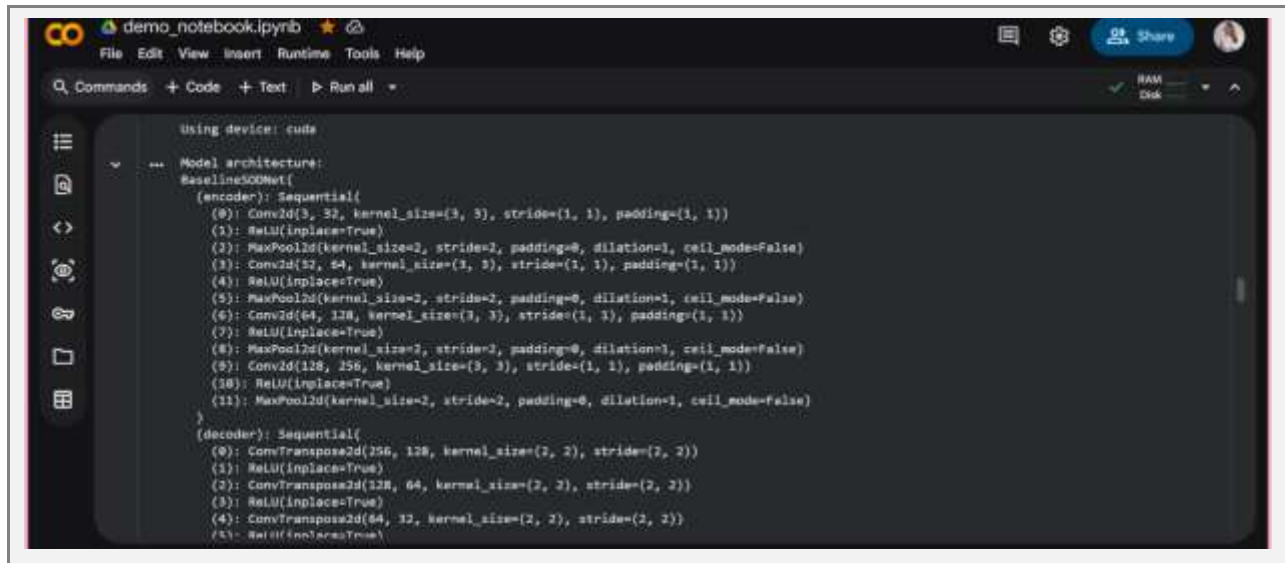
The model used in this project is a CNN encoder-decoder architecture built from scratch in PyTorch. I did not use any pretrained models, pretrained weights, or pretrained feature extractors.

The baseline model takes an RGB image with shape 3 x 128 x 128 as input. The encoder part contains convolutional layers with ReLU activations, followed by MaxPooling layers. The purpose of the encoder is to extract useful features from the image while gradually reducing the spatial size.

The decoder part uses ConvTranspose2D layers with ReLU activations. These layers upsample the feature maps back toward the original image resolution. The final layer produces a one-channel output mask with the same height and width as the input image. A sigmoid activation is used at the end so the output values are between 0 and 1.

The baseline architecture follows the project requirements: 4 Conv2D encoder blocks with ReLU and MaxPooling, 4 ConvTranspose2D decoder layers with ReLU, and a 1-channel sigmoid output mask.

Before training, I tested one forward pass using a batch from the DataLoader. The model successfully produced an output with shape [8, 1, 128, 128], which confirmed that the architecture was compatible with the dataset and masks.



```
Using device: cuda

Model architecture:
BaselineSODNet(
  (encoder): Sequential(
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (9): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (10): ReLU(inplace=True)
    (11): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (decoder): Sequential(
    (0): ConvTranspose2d(256, 128, kernel_size=(2, 2), stride=(2, 2))
    (1): ReLU(inplace=True)
    (2): ConvTranspose2d(128, 64, kernel_size=(2, 2), stride=(2, 2))
    (3): ReLU(inplace=True)
    (4): ConvTranspose2d(64, 32, kernel_size=(2, 2), stride=(2, 2))
  )
)
```

Figure 4. Forward pass test showing that the model input and output shapes are correct.

5. Training Method

The model was trained in Google Colab using a T4 GPU. I used PyTorch and wrote a full custom training loop instead of relying on a high-level training framework. Each training epoch included the forward pass, loss calculation, backward pass, optimizer update, validation step, metric logging, and checkpoint saving.

The optimizer used was Adam with a learning rate of 0.001. Adam was chosen because it is commonly used for deep learning projects and usually works well as a starting optimizer.

The loss function was Binary Cross Entropy + 0.5 * (1 - IoU). Binary Cross Entropy helps the model learn pixel-wise binary classification, where each pixel is either salient or background. The IoU part helps the model focus on the overlap between the predicted mask and the ground-truth mask.

Training was configured for 20 epochs, which is within the project requirement of 15-25 epochs. Early stopping was used to stop training when the validation loss stopped improving. The baseline model stopped early at epoch 11 because early stopping was triggered.

The best model checkpoint was saved automatically during training. The project also saved the latest checkpoint, which makes it possible to resume training if Colab disconnects.

```
Current folder: /content/drive/MyDrive/500_Project
Baseline checkpoint exists: True
Using device: cuda
test: 100% 19/19 [00:01<00:00, 10.86it/s]
Test metrics: {'iou': 0.3412478401472694, 'precision': 0.49516892433166504, 'recall': 0.5923948585987091, 'f1': 0.5168370745683971, 'mae': 0.1111111111111111}
Saved metrics to outputs/metrics/baseline_test_metrics.csv
Saved visualization: outputs/visualizations/baseline_sample_1.png
Saved visualization: outputs/visualizations/baseline_sample_2.png
Saved visualization: outputs/visualizations/baseline_sample_3.png
Saved visualization: outputs/visualizations/baseline_sample_4.png
Saved visualization: outputs/visualizations/baseline_sample_5.png
```

Figure 5. Baseline training output showing validation results, checkpoint saving, and early stopping at epoch 11.

6. Evaluation Metrics

After training, the models were evaluated on the test set. I used several metrics because a single metric does not always describe segmentation performance completely.

? Intersection over Union (IoU): measures the overlap between the predicted mask and the ground-truth mask. A higher IoU means better overlap.

? Precision: measures how many predicted salient pixels were actually correct. High precision means fewer false positive salient regions.

? Recall: measures how many ground-truth salient pixels were found by the model. High recall means the model missed fewer salient regions.

? F1-score: combines precision and recall into one balanced score.

? Mean Absolute Error (MAE): measures the average pixel-level difference between the predicted mask and the ground-truth mask. Lower MAE is better.

These metrics were calculated on the test set after training. I also generated visualizations because image segmentation results should be inspected visually, not only through numbers.

7. Experiments and Results

After the baseline model was working, I trained an improved model for the experiments section. The improved model was still built from scratch and did not use any pretrained weights.

The improved model included three main changes: Batch Normalization, Dropout, and deeper convolutional blocks. Batch normalization was added to help stabilize training. Dropout was added to reduce overfitting. Deeper convolutional blocks were added so the model could learn more complex features.

Both the baseline model and the improved model were evaluated on the same test set using IoU, Precision, Recall, F1-score, and MAE. The comparison table below shows the results.

	model	iou	precision	recall	f1	mae
0	baseline	0.341248	0.495169	0.592395	0.516837	0.306338
1	improved	0.537026	0.667645	0.750282	0.696083	0.202787

Figure 6. Baseline vs improved model comparison using IoU, Precision, Recall, F1-score, and MAE.

The comparison helped show how architectural changes can affect performance. If the improved model does not perform better in every metric, that is still useful information. In deep learning, adding more layers or regularization does not always guarantee better results, especially on a small dataset or with limited training time. The experiment still satisfies the goal because it compares a baseline model with an intentionally modified version and analyzes the effect of the changes.

In addition to numerical metrics, I generated visual predictions for both models. Each visualization includes the input image, ground-truth mask, predicted mask, and overlay.



Figure 7. Baseline model visualization showing input image, ground-truth mask, predicted mask, and overlay.

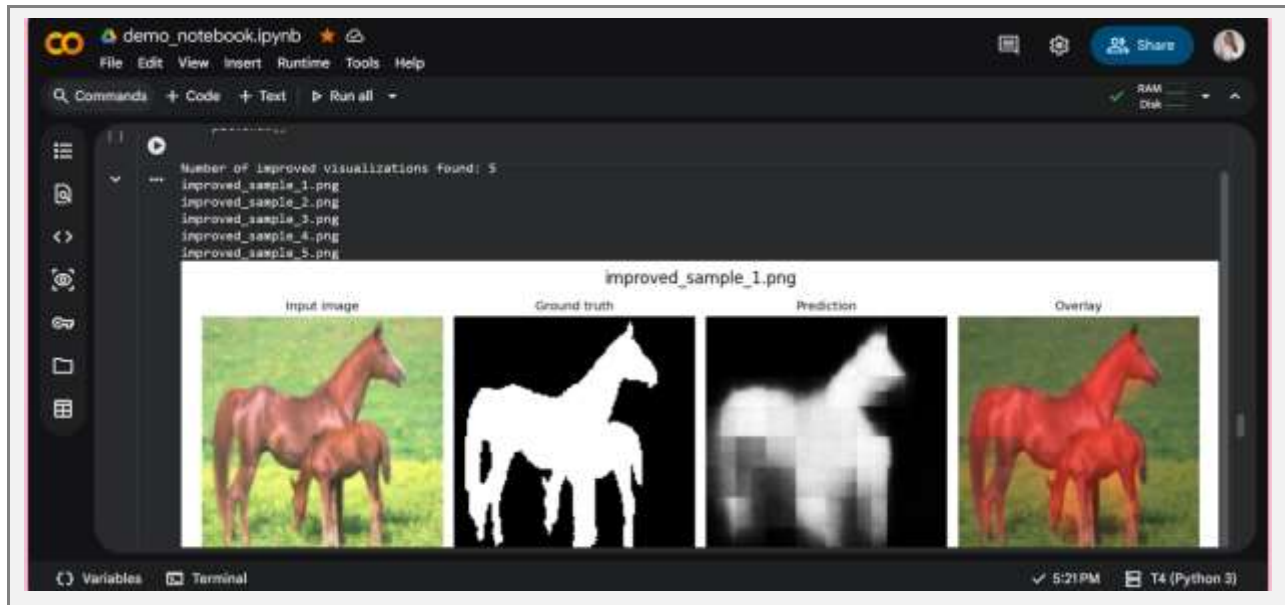


Figure 8. Improved model visualization showing input image, ground-truth mask, predicted mask, and overlay.

The visual results are useful because they show where the model correctly detects the salient object and where it struggles. In some cases, the model may capture the general object region but miss fine boundaries. This is expected for a simple CNN encoder-decoder trained from scratch at 128x128 resolution.

8. Demo

A simple demo was created in Google Colab.

The demo shows the input image, the predicted saliency mask, the overlay visualization, and the inference time per image. The inference time was measured for a single uploaded image.

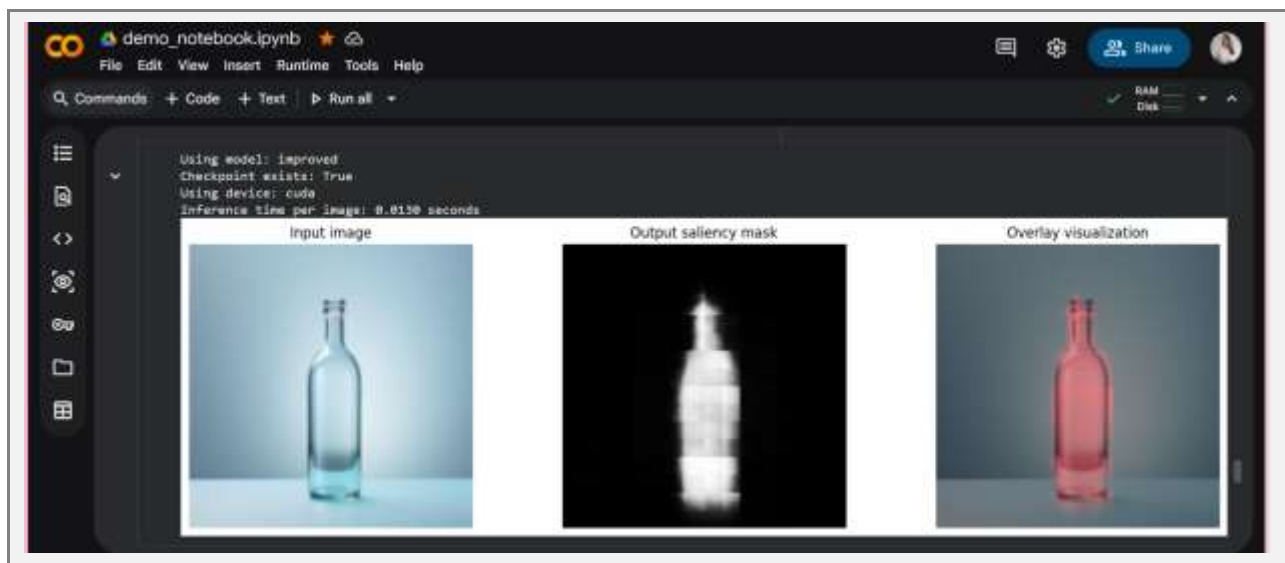


Figure 9. Demo output showing the uploaded image, predicted saliency mask, overlay visualization, and inference time.

The demo confirms that the trained model can be used on a new image outside the training and test visualizations.

9. Learnings and Challenges

This project helped me understand the complete deep learning pipeline more clearly. Before training the model, I had to organize the dataset, check image-mask pairs, resize images, normalize values, split the dataset, and test the DataLoader. These steps were just as important as building the model.

One important lesson was that visual inspection is necessary in segmentation tasks. If the masks are wrong, mismatched, or incorrectly resized, the model may still run but the results will not be meaningful. Visualizing the dataset and predictions helped me catch potential issues early.

Another learning point was understanding the encoder-decoder structure. The encoder reduces the image into feature maps, while the decoder upsamples those features into a mask. Testing a forward pass before training helped confirm that the model output shape matched the ground-truth mask shape.

A challenge was working with Colab and training time. Since Colab sessions can disconnect, checkpoint saving was important. Another challenge was interpreting results. Numerical metrics are helpful, but the predicted masks and overlays give a clearer understanding of what the model is actually learning.

The improved model experiment also showed that adding more techniques does not always automatically improve every result. Batch normalization, dropout, and deeper blocks can help, but their effect depends on dataset size, training settings, and model behavior.

10. Conclusion

In this project, I successfully built an end-to-end Salient Object Detection system from scratch. The project used the ECSSD dataset, PyTorch, and Google Colab with a T4 GPU. The full pipeline included dataset preparation, preprocessing, augmentation, CNN encoder-decoder model design, training, evaluation, visualization, and demo inference.

The baseline model was trained using Binary Cross Entropy plus IoU loss and Adam optimizer. Early stopping was used, and the best model checkpoint was saved. An improved model was also trained using batch normalization, dropout, and deeper convolutional blocks. Both models were evaluated using IoU, Precision, Recall, F1-score, and MAE.

The final demo shows that the trained model can take an input image and produce a saliency mask with an overlay visualization and inference time. Overall, this project gave me practical experience with building a complete deep learning project from beginning to end.